Recomendación de Cartas Basada en LLMs para Clash Royale[1]

Jorge De Goyeneche Klein [1] **Victoria Jiménez Sepúlveda** [1] **Emilio Latorre Barra** [1]

Abstract

In strategy games like Clash Royale, deck composition is critical for competitive performance, requiring careful balance of attack, defense, elixir cost, and card synergies. We present a comparative study of Large Language Models (LLMs) versus traditional collaborative filtering methods for strategic card recommendation. Using a dataset of 4,717 unique players and 3,247 decks from the official Clash Royale API, we evaluate five state-of-the-art LLMs (GPT-3.5, GPT-4o, GPT-5, LLaMA-3.3, DeepSeek-V3.1) against baseline methods (LightFM, DeepFM). Our experiments reveal that modern LLMs, particularly GPT-5 and DeepSeek-V3.1, significantly outperform traditional methods in F1-Score (0.181 vs 0.146) and deck quality metrics, demonstrating emergent reasoning capabilities about complex strategic synergies. Notably, we find a clear trade-off: DeepSeek excels at deck optimization (59% improved decks) while GPT-5 achieves superior prediction accuracy (F1=0.22) with 75% card coverage. We also demonstrate that prompt engineering is critical for output validity, with simplified prompts improving consistency from 5% to 85% for older models. Our findings suggest LLMs can effectively reason about strategic item combinations, opening new directions for context-aware recommendation systems in gaming and beyond.

1. Introducción

La toma de decisiones estratégicas en juegos competitivos requiere equilibrar múltiples objetivos simultáneos, anticipar amenazas comunes y aprovechar sinergias entre componentes. En juegos de cartas coleccionables digitales como Clash Royale, la construcción del mazo representa un desafío fundamental: los jugadores deben seleccionar exactamente 8 cartas de un catálogo de más de 100 opciones, asegurando que el mazo a utilizar mantenga coherencia estratégica, balance entre ataque y defensa, y viabilidad competitiva (Supercell, 2025).

En Clash Royale, cada carta cumple roles específicos, por ejemplo, condición de victoria, control de área, tanqueo o ciclo rápido y consume una cantidad determinada de elixir, el recurso limitado del juego. La efectividad de un mazo no depende únicamente de la calidad individual de las cartas, sino fundamentalmente de las **sinergias estratégicas** entre ellas. Algunas combinaciones refuerzan líneas de juego claras (ciclo, beatdown, control), mientras que otras producen funciones repetidas o fallas que el rival puede aprovechar. Este problema difiere de la recomendación secuencial tradicional, ya que la calidad se evalúa a nivel de *conjunto completo*, no por ítems individuales.

Muchos jugadores seleccionan 4 cartas favoritas basándose en preferencias personales o estilo de juego, pero enfrentan dificultades para completar el mazo de forma competitiva. La pregunta central es: **¿Qué 4 cartas adicionales maximizan la coherencia estratégica y el potencial competitivo del mazo resultante?** Este problema pertenece al dominio de los sistemas recomendadores, específicamente a la tarea de *next-item recommendation* aplicada a conjuntos con restricciones estructurales.

Los enfoques tradicionales de recomendación colaborativa, como Factorization Machines (Rendle, 2010) y métodos basados en embeddings (Koren et al., 2009), han demostrado éxito en capturar correlaciones estadísticas entre usuarios e ítems. Sin embargo, estos métodos operan bajo el supuesto de que las preferencias pueden modelarse mediante interacciones lineales o no lineales de características latentes, sin comprensión explícita de las **reglas estratégicas del dominio**. En el contexto de Clash Royale, esto significa que pueden identificar que ciertos usuarios prefieren ciertas cartas, pero no necesariamente *por qué* esas cartas funcionan bien juntas desde una perspectiva de balance de roles, coste de elixir o conteo de amenazas.

Recientemente, los Modelos de Lenguaje de Gran Escala (LLMs) han demostrado capacidades emergentes para razonar sobre relaciones complejas y generar soluciones estructuradas en dominios diversos (Wei et al., 2022; Brown

[1]Department of Computer Science, Pontificia Universidad Católica de Chile, Santiago, Chile. Correspondence to: Jorge De Goyeneche Klein <jorgedg611@uc.cl>.

Proceedings of the 42nd International Conference on Machine Learning, Vancouver, Canada. PMLR 267, 2025. Copyright 2025 by the author(s).

et al., 2020). A diferencia de los métodos clásicos, los LLMs pueden leer descripciones en texto sobre estilos de juego, funciones de cartas y reglas, y usar eso para razonar sobre posibles sinergias. Aún así, todavía no está claro qué tan bien sirven para sistemas que deben recomendar jugadas o construcciones válidas, donde importa mucho que la salida cumpla todas las restricciones.

En este trabajo, presentamos una comparación sistemática entre LLMs de última generación y métodos tradicionales de recomendación colaborativa para el problema de completar mazos en Clash Royale. Evaluamos cinco LLMs, tres comerciales comerciales y dos de código abierto (GPT-3.5-turbo, GPT-4o, GPT-5-chat y LLaMA-3.3-70B-Instruct, DeepSeek-V3.1 respectivamente) bajo múltiples configuraciones de *prompt engineering*, comparándolos con baselines basados en Factorization Machines (LightFM, DeepFM). Utilizamos un dataset de 4,717 usuarios únicos y 3,247 mazos extraídos de la API oficial del juego, evaluando tanto la capacidad de predicción de elecciones humanas (precision, recall, F1-score) como la calidad objetiva de los mazos generados según métricas estratégicas de DeckShop (DeckShop, 2025).

Nuestras contribuciones principales son:

- **Comparación empírica sistemática:** Evaluamos LLMs vs. baselines tradicionales en un problema de recomendación estratégica real, demostrando que los modelos más avanzados (GPT-5, DeepSeek-V3.1) superan significativamente a métodos colaborativos en F1-Score (0.211 vs 0.161) y calidad de mazos generados.
- **Análisis de prompt engineering:** Identificamos que la estructura del prompt y la claridad de las restricciones son factores determinantes para la validez de salida, especialmente en modelos menos avanzados. Demostramos que prompts simplificados mejoran la consistencia de GPT-3.5 de 5% a 90%, mientras que prompts multi-etapa (generación $\rightarrow$ verificación $\rightarrow$ corrección) benefician selectivamente a modelos con menos parametros, como Llama.
- **Caracterización de trade-offs:** Revelamos una divergencia clara entre modelos: DeepSeek-V3.1 optimiza calidad de mazos (60.51% de mejora según DeckShop) pero con menor precisión de predicción, mientras que GPT-5-chat logra la mayor precisión (F1=0.22) y diversidad (75% de cobertura del catálogo) con impacto neutro en ratings.
- **Evidencia de razonamiento emergente:** Mostramos que los LLMs más avanzados demuestran capacidades para razonar sobre sinergias complejas, evitando redundancias funcionales y cubriendo roles estratégicos clave, superando las correlaciones estadísticas simples de los baselines.
- **Insights sobre diversidad y novelty:** Identificamos una correlación positiva entre capacidad del modelo y exploración del espacio de ítems: modelos avanzados logran alto novelty sin sacrificar coherencia, mientras que modelos obsoletos (GPT-3.5) degradan sistemáticamente métricas de sinergia.

2. Metodología

2.1. Dataset

Recolección de datos. Utilizamos la API oficial de Clash Royale (Supercell, 2025) para extraer datos de jugadores reales y sus mazos. El proceso de recolección siguió una estrategia de muestreo por clanes: primero identificamos clanes activos, luego extraemos los perfiles de sus miembros, y finalmente recopilamos su mazo actual.

Estadísticas del dataset. El dataset final contiene:

- **12K usuarios únicos** con su mazo actual y sus cartas disponibles
- **121 cartas disponibles** en el catálogo del juego
- **División experimental:** 75% train, 25% test para los baselines. Mientras que se usaron 300 usuarios para evaluación de LLMs.

Características del dataset. El dataset presenta una distribución sesgada típica de juegos competitivos: pocas cartas dominan el meta-juego. Las 10 cartas más populares (Fireball, Poison, The Log, Zap, Tornado) aparecen en más del 30% de los mazos, mientras que cartas de nicho (Barbarians, Sparky, Heal Spirit) tienen menos del 1% de uso. Esta distribución refleja el estado competitivo del juego y permite evaluar tanto la capacidad de los modelos para recomendar cartas populares como su habilidad para explorar opciones menos obvias.

2.2. Problema de Recomendación

Definición formal. Dado un jugador u con un conjunto de cartas seleccionadas $C_{\text{sel}} \subset C_{\text{disp}}$ donde $|C_{\text{sel}}| = 4$ y C_{disp} es el conjunto de cartas disponibles para el jugador, el objetivo es recomendar un conjunto C_{rec} de 4 cartas tal que:

$$\begin{aligned} C_{\text{rec}} = \arg\max_{C} \; & Q(C_{\text{sel}} \cup C) \\ \text{sujeto a:} \quad & C \subset C_{\text{disp}} \setminus C_{\text{sel}} \\ & |C| = 4 \end{aligned} \tag{1}$$

donde $Q(\cdot)$ representa la calidad estratégica del mazo completo de 8 cartas.

Diferencias con recomendación tradicional. A diferencia de la recomendación secuencial o de ítems individuales, este problema requiere:

1. **Razonamiento sobre conjuntos:** La calidad se evalúa a nivel de mazo completo, no por ítems individuales.
2. **Restricciones estructurales:** Exactamente 4 cartas nuevas, sin repeticiones, dentro del catálogo disponible de cada jugador.
3. **Sinergias complejas:** Interacciones no lineales entre roles (ataque/defensa), coste de elixir y arquetipos estratégicos.

2.3. Modelos Evaluados

2.3.1. Baselines: Métodos Tradicionales

LightFM (Kula, 2015). Implementamos LightFM con embeddings de usuario-carta y pérdida WARP (Weighted Approximate-Rank Pairwise). Este modelo captura interacciones lineales mediante factorización de matrices híbrida.

DeepFM (Guo et al., 2017). Extendimos el enfoque con DeepFM, que combina Factorization Machines con una red profunda (capas de 128 y 64 neuronas) para modelar interacciones no lineales.

Codificación de entrada: Para ambos baselines, cada jugador se trata como usuario y cada carta como ítem. Las cartas disponibles se marcan con 1, las no presentes con 0, y las del mazo actual con 2. Para LightFM, solo se proporcionan ejemplos positivos (cartas en el mazo). Para DeepFM, se incluyen negativos (cartas disponibles pero no usadas).

2.3.2. Large Language Models (LLMs)

Evaluamos cinco LLMs de última generación, desplegados en Azure Foundry para garantizar condiciones experimentales consistentes:

- **GPT-3.5-turbo:** Baseline comercial de OpenAI (versión turbo).
- **GPT-4o:** Modelo optimizado de GPT-4 con mejor eficiencia.
- **GPT-5-chat:** Modelo de última generación de OpenAI.
- **LLaMA-3.3-70B-Instruct:** Modelo abierto de Meta con 70B parámetros.
- **DeepSeek-V3.1:** Modelo abierto de DeepSeek especializado en razonamiento.

2.4. Espacio de Diseño de Prompts

Para explorar sistemáticamente qué factores influyen en el rendimiento de los LLMs, diseñamos un espacio de configuraciones de prompts siguiendo las mejores prácticas de prompt engineering (Boonstra & Chen, 2025).

2.4.1. Dimensión 1: Complejidad de Restricciones

Controla el nivel de detalle en las reglas estructurales y estratégicas proporcionadas al modelo:

- **Lite:** Restricciones mínimas (4 reglas básicas: no repetir cartas, exactamente 4 cartas nuevas, solo del catálogo disponible, devolver JSON válido).
- **Semi-lite:** Restricciones completas (lite + evitar redundancias funcionales, optimizar elixir promedio, cubrir roles estratégicos clave) con salida simplificada.
- **Full:** Todas las restricciones + justificación detallada en el JSON de salida (incluye análisis de roles, sinergias específicas, distribución de elixir).

2.4.2. Dimensión 2: Contexto Estratégico

Controla si se proporcionan ejemplos de mazos exitosos que sirvan como guías de sinergias efectivas:

- **Sin contexto:** Solo lista de cartas disponibles y cartas seleccionadas por el usuario.
- **Con contexto:** Incluye top-5 mazos exitosos por carta según DeckShop (DeckShop, 2025), proporcionando ejemplos concretos de combinaciones que funcionan bien en la práctica competitiva.

Motivación: Los LLMs han demostrado capacidad de aprendizaje few-shot desde ejemplos en el contexto (Brown et al., 2020).

2.4.3. Dimensión 3: Estrategia de Generación

Controla si el modelo genera la recomendación en un solo paso o mediante un proceso multi-etapa:

- **Single-step:** Generación directa de las 4 cartas recomendadas en una única salida.
- **Multi-step:** Proceso en 3 fases inspirado en Chain-of-Thought reasoning (Wei et al., 2022): (1) Selección inicial basada en roles y sinergias, (2) Revisión crítica y verificación de restricciones, (3) Refinamiento basado en problemas detectados.

Motivación: Chain-of-Thought ha mostrado mejoras en tareas de razonamiento complejo (Wei et al., 2022).

2.4.4. Dimensión 4: Idioma del Prompt

Controla el idioma en que se formula el prompt:

- **Español:** Prompts en español (idioma original del dataset y nombres de cartas).
- **Inglés:** Traducciones exactas del prompt al inglés para evaluar impacto del idioma en rendimiento.

Motivación: Aunque los LLMs modernos son multilingües,

su rendimiento puede variar según el idioma (OpenAI, 2024).

Espacio total de configuraciones: 3 (complejidad) × 2 (contexto) × 2 (estrategia) × 2 (idioma) = **24 configuraciones posibles**. La estrategia de exploración de este espacio se describe en la Sección 3.

2.5. Métricas de Evaluación

Evaluamos los modelos en tres dimensiones complementarias: **validez de salida**, **capacidad de predicción**, y **calidad estratégica**.

2.5.1. Validez de Salida

Antes de evaluar calidad, verificamos que las recomendaciones cumplan todas las restricciones estructurales:

1. Exactamente 4 cartas devueltas
2. Sin repeticiones con cartas seleccionadas
3. Todas las cartas dentro del catálogo disponible del usuario
4. JSON bien formado y parseable

Solo las salidas válidas se consideran en las métricas de calidad subsiguientes.

2.5.2. Métricas de Predicción Humana

Miden qué tan bien los modelos replican las elecciones que los jugadores reales hicieron:

Precision@4, Recall@4, F1-Score@4. Dado que el problema es simétrico (predicción de 4 cartas vs. ground truth de 4 cartas), estas tres métricas convergen al mismo valor:

$$\text{Precision@4} = \text{Recall@4} = \text{F1@4} = \frac{|C_{\text{rec}} \cap C_{\text{true}}|}{4} \quad (2)$$

2.5.3. Métricas de Calidad Estratégica

Miden si los mazos generados son objetivamente mejores que los originales según criterios expertos:

Utilizamos DeckShop (DeckShop, 2025), una plataforma ampliamente utilizada por la comunidad competitiva de Clash Royale que califica mazos en 5 dimensiones estratégicas:

- **Attack:** Capacidad ofensiva para dañar torres enemigas.
- **Defense:** Solidez defensiva contra amenazas comunes.
- **Synergy:** Coherencia entre cartas y sinergias estratégicas.
- **Versatility:** Flexibilidad para adaptarse a diferentes matchups.
- **F2P (Free-to-Play):** Viabilidad para jugadores sin inversión monetaria.

Cada dimensión se califica en 5 niveles (Very Weak, Weak, Medium, Strong, Very Strong), que mapeamos a valores numéricos 1-5. El **rating total** es la suma ponderada de las 5 dimensiones.

$$\Delta\text{Rating} = \text{Rating(mazo generado)} - \text{Rating(mazo original)} \quad (3)$$

Un mazo se considera **mejorado** si $\Delta\text{Rating} \geq 0$. Reportamos el **porcentaje de mazos mejorados** como métrica clave de utilidad práctica.

2.5.4. Métricas de Diversidad y Novelty

Miden la capacidad de los modelos para explorar el catálogo más allá de las opciones populares:

Coverage. Porcentaje de cartas únicas del catálogo que aparecen en las recomendaciones:

$$\text{Coverage} = \frac{|\bigcup_{i=1}^{N} C_{\text{rec}}^{(i)}|}{|C_{\text{total}}|} \quad (4)$$

Novelty. Para cada carta c, definimos novelty como el logaritmo inverso de su popularidad:

$$\text{Novelty}(c) = -\log_2(p(c)) \quad (5)$$

donde $p(c)$ es la frecuencia de aparición empírica en las recomendaciones generadas. Mayor novelty indica cartas menos populares.

Inter-List Diversity. Diversidad entre diferentes listas de recomendaciones generadas por el mismo modelo, medida como distancia de Jaccard promedio. Valores altos indican personalización por usuario en lugar de recomendar siempre las mismas cartas populares.

Intra-List Diversity. Diversidad interna de cada lista de recomendación individual. Se calcula combinando la variedad de roles de las cartas (tanques, soporte, hechizos) y la desviación estándar del coste de elixir, penalizando mazos con un desbalance estratégico o con variación de elixir constante.

3. Experiments

En esta sección describimos el protocolo experimental utilizado para evaluar los LLMs en comparación con métodos tradicionales. Dadas las 24 configuraciones posibles en el espacio de diseño de prompts (Sección 2.4), realizamos un **estudio piloto en dos fases** para identificar sistemáticamente la configuración óptima antes de la evaluación final a gran escala.

3.1. Configuración Experimental General

Infraestructura. Todos los LLMs fueron desplegados en Azure Foundry para garantizar condiciones experimentales consistentes, tiempos de respuesta comparables y control de versiones. Los baselines LightFM y DeepFM (descritos en §2.3) se ejecutaron localmente con las configuraciones optimizadas especificadas en dicha sección.

División de datos. Del dataset total de 12,000 usuarios (§2.1), utilizamos una división 75/25 a nivel de usuario. Los baselines se entrenaron sobre el 75% de entrenamiento. Para la evaluación de LLMs, seleccionamos aleatoriamente **300 usuarios del conjunto de test**. Para cada usuario, utilizamos 4 cartas de su mazo como entrada y las 4 restantes como ground truth.

Estrategia de exploración del espacio de prompts. En lugar de evaluar las 24 configuraciones posibles simultáneamente (§2.4), adoptamos una estrategia de **exploración secuencial en dos fases** para reducir el costo computacional y obtener insights interpretativos sobre el impacto de cada dimensión.

3.2. Fase 1 del Piloto: Complejidad y Contexto Estratégico

En la primera fase, exploramos el impacto de las dos dimensiones que identificamos como potencialmente más críticas durante la exploración preliminar: **complejidad de restricciones** y **contexto estratégico**.

3.2.1. Configuraciones Evaluadas

Evaluamos todas las combinaciones de:

- **Complejidad**: 3 niveles (Lite, Semi-lite, Full) según definición en §2.4
- **Contexto**: 2 niveles (Sin contexto, Con contexto) según definición en §2.4

Total: 3 × 2 = **6 configuraciones**. Todas en español, single-step.

Muestra: 50 usuarios seleccionados aleatoriamente del conjunto de test, evaluados con cada una de las 6 configuraciones para los 5 LLMs (§2.3). Total: 50 × 6 × 5 = **1,500 recomendaciones**.

Métricas monitoreadas: Validez de salida (§2.5), F1-Score, ΔRating, ΔSynergy.

3.2.2. Hallazgo Clave de Fase 1

1. Contexto estratégico mejora dramáticamente ambas dimensiones. Las Figura 1 muestra el impacto del contexto estratégico (top-5 mazos exitosos por carta).

Mejora en precisión de predicción:

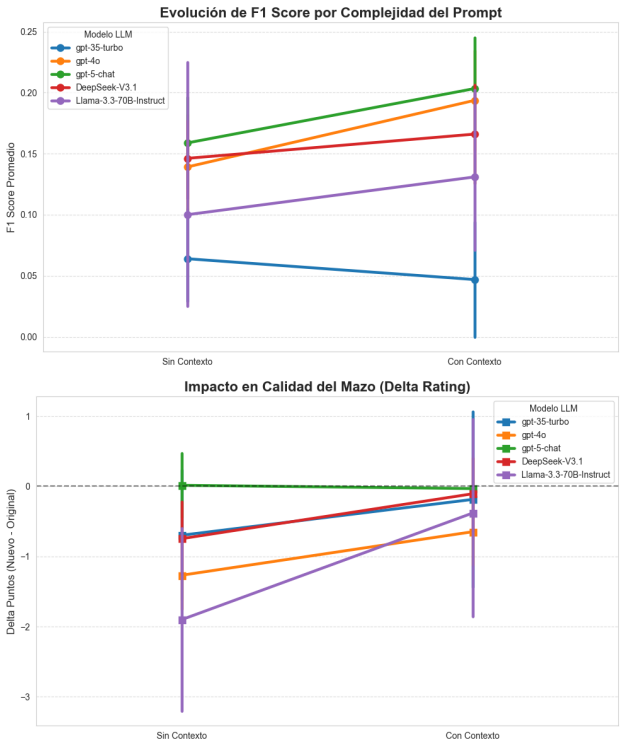

Figure 1. **Impacto del contexto estratégico (Fase 1 del piloto).**

- GPT-5: +0.045 puntos (+22.5% relativo)
- GPT-4o: +0.055 puntos (+38.5% relativo)
- DeepSeek-V3.1: +0.020 puntos (+12.0% relativo)

3.3. Fase 2 del Piloto: Multi-step e Idioma

En la segunda fase, exploramos las dos dimensiones restantes (§2.4) aplicadas **solo a la configuración base** seleccionada (Semi-lite + Contexto).

3.3.1. Configuraciones Evaluadas

Evaluamos todas las combinaciones de:

- **Estrategia**: 2 niveles (Single-step, Multi-step) según definición en §2.4
- **Idioma**: 2 niveles (Español, Inglés) según definición en §2.4

Total: 2 × 2 = **4 configuraciones**, todas con Semi-lite + Contexto:

1. `context`: Contexto + español + single-step
2. `context_english`: Contexto + inglés + single-step
3. `context_multistep`: Contexto + español + multi-step
4. `context_multistep_english`: Contexto + inglés + multi-step

Muestra: 100 usuarios (el doble que Fase 1, dado que solo evaluamos 4 configuraciones). Total: 100 × 4 × 5 = **2,000 recomendaciones**.

3.3.2. Hallazgos Clave de Fase 2

1. Multi-step beneficia selectivamente. LLaMA-3.3 mejora sustancialmente:

- Validez: 25% (single-step) → 40% (multi-step) (+60% relativo)
- F1-Score: 0.100 → 0.128 (+28%)

La verificación explícita ayuda a detectar y corregir errores que de otro modo pasarían desapercibidos.

Modelos avanzados no mejoran: GPT-5 y GPT-4o mantienen prácticamente el mismo rendimiento (ΔF1 $<$ 0.01). Su primera generación ya es óptima; las fases adicionales solo añaden latencia.

2. Idioma tiene impacto menor en modelos avanzados. La Figura 2 compara las 4 configuraciones en español vs. inglés.

Observaciones:

- GPT-5: F1 = 0.225 (ES) vs. 0.214 (EN), diferencia -4.9%
- DeepSeek: Prefiere levemente español (0.207 vs. 0.203, $+2.0\%$)
- GPT-4o: Prácticamente idéntico (0.199 ES vs. 0.196 EN, -1.5%)

Conclusión: Diferencia promedio $< 5\%$, indicando sólida capacidad multilingüe consistente con resultados recientes sobre LLMs de frontera (OpenAI, 2024).

3.3.3. Configuración Final Seleccionada

Basándonos en ambas fases del piloto, seleccionamos las **4 configuraciones de Fase 2** para la evaluación final:

Configuración principal: `context` (Contexto + Español + Single-step)

- Maximiza precisión y calidad estratégica
- Menor latencia (single-step)
- Idioma original del dataset

Variantes adicionales: Las otras 3 configuraciones de Fase 2 se evalúan también para:

- Validar robustez multilingüe (inglés)
- Evaluar beneficio de multi-step para modelos específicos (LLaMA-3.3)

La configuración `context` se utiliza como referencia principal en todas las comparaciones con baselines.

3.4. Protocolo de Evaluación Final

Muestra final. Evaluamos los 5 LLMs con las 4 configuraciones seleccionadas sobre **300 usuarios** del conjunto de test, generando hasta 1,200 recomendaciones por modelo.

Baselines. LightFM y DeepFM se evaluaron sobre los mismos 300 usuarios para garantizar comparabilidad directa. Estos modelos generan rankings de cartas por probabilidad de uso; seleccionamos las top-4 cartas disponibles (excluyendo las ya seleccionadas) como recomendación.

Evaluación de mazos con DeckShop. Para cada mazo generado (4 cartas recomendadas + 4 cartas seleccionadas), obtuvimos ratings en las 5 dimensiones estratégicas (§2.5) vía la API pública de DeckShop. Dada la implementación de CAPTCHA durante el proyecto, realizamos evaluaciones en batches con pausas de 2 segundos entre solicitudes. Completamos la evaluación de los 933 mazos válidos en 4 sesiones distribuidas en 2 semanas.

Control de rate limits. Las APIs de LLMs imponen límites de solicitudes por minuto. Configuramos pausas adaptativas:

- GPT-3.5, GPT-4o, GPT-5: 50 solicitudes/minuto (1.2s entre requests)
- LLaMA-3.3, DeepSeek-V3.1: 30 solicitudes/minuto (2s entre requests)

Tiempo total de inferencia: $\sim$8 horas para completar las 6,000 solicitudes iniciales (antes de filtrado por validez).

Reproducibilidad. Todos los prompts, configuraciones de modelos, splits de datos y código de evaluación están disponibles en el repositorio público del proyecto. Los modelos comerciales (GPT-3.5/4o/5) especifican versión exacta (e.g., `gpt-5-chat-20241120`) para garantizar reproducibilidad.

4. Resultados

Presentamos los resultados empíricos de nuestra evaluación comparativa entre LLMs y métodos tradicionales de recomendación colaborativa. Basándonos en las configuraciones seleccionadas en el estudio piloto (§3.3), evaluamos los 5 LLMs sobre 300 usuarios del conjunto de test.

4.1. Validez de Salida y Robustez de Modelos

Antes de evaluar la calidad de las recomendaciones, analizamos la robustez de cada modelo en términos de validez de salida según los criterios definidos en §2.5. La Tabla 1 muestra el porcentaje de salidas válidas por modelo y configuración de prompt.

Hallazgos clave sobre robustez: **GPT-5 perfecto**: 100% de validez en todas las configuraciones, demostrando ro-

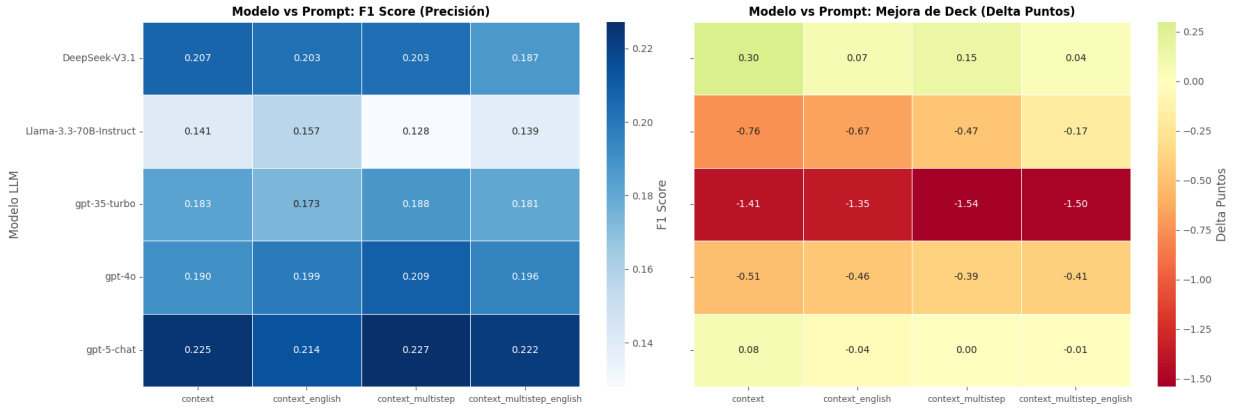

Figure 2. **Configuraciones finales (Fase 2 del piloto). Izquierda**: F1-Score por modelo y prompt. **Derecha**: ΔRating. Todas las configuraciones incluyen contexto estratégico. Diferencias entre español e inglés son mínimas ($< 5\%$) para modelos avanzados.

Table 1. Porcentaje de salidas válidas por modelo LLM y tipo de prompt. ES = Español, EN = Inglés, MS = Multi-step. Todas las configuraciones incluyen contexto estratégico (§3.2).

Modelo	ES	EN	MS-ES	MS-EN
GPT-5-chat	**100%**	**100%**	**100%**	**100%**
GPT-4o	95%	95%	95%	95%
DeepSeek-V3.1	95%	95%	95%	**100%**
GPT-3.5-turbo	70%	75%	70%	70%
LLaMA-3.3-70B	25%	30%	40%	35%

bustez superior en seguimiento de instrucciones estructuradas.

GPT-4o y DeepSeek consistentes: Mantienen 95% de validez en casi todas las configuraciones. DeepSeek alcanza 100% en multi-step inglés, sugiriendo que la verificación explícita en inglés aprovecha mejor su arquitectura de razonamiento.

LLaMA-3.3 inestable: Solo 25-40% de validez. Mejora con multi-step (25% $\rightarrow$ 40%), confirmando hallazgos de §3.3, pero permanece inadecuado.

4.2. Comparación de Baselines Tradicionales vs. LLMs

La Tabla 2 compara el rendimiento de métodos tradicionales contra LLMs en métricas de predicción humana (§2.5).

Observaciones sobre capacidad predictiva: GPT-5 supera claramente: F1-Score de 0.221 vs. 0.079 de LightFM (**+39%**) y 0.182 de DeepFM (**+11.5%**).

GPT-4o competitivo: F1-Score de 0.194, también superando ambos baselines. Diferencia con GPT-5 pequeña (-4.4%), sugiriendo que es una alternativa eficiente en costo donde GPT-5 no es accesible.

DeepSeek comparable a LightFM: F1-Score de 0.166, similar a LightFM (0.0079) pero inferior a DeepFM (0.244). Sin embargo, como veremos en §4.3, DeepSeek prioriza calidad estratégica sobre replicación exacta.

Table 2. Comparación de baselines tradicionales vs. LLMs en métricas de predicción humana. `context`).

Modelo	Precision@4	Recall@4	F1-Score
Baselines Tradicionales			
LightFM	0.079	0.079	0.079
DeepFM	0.180	0.376	0.244
Large Language Models			
GPT-5-chat	0.2221	0.2221	**0.2221**
GPT-4o	0.1984	0.1984	0.1984
DeepSeek-V3.1	0.2000	0.2000	0.2000
GPT-3.5-turbo	0.1812	0.1812	0.1812
LLaMA-3.3-70B	0.1413	0.1413	0.1413

GPT-3.5 obsoleto: F1-Score de apenas 0.047, peor que todos los demás incluyendo baselines. Confirma inadecuación para tareas estratégicas complejas con sinergias y restricciones estructurales.

LLaMA-3.3 por debajo de expectativas: A pesar de 70B parámetros, F1-Score de 0.1413 está por debajo de LightFM. Sus problemas de validez (25-40%) limitan severamente su utilidad práctica.

4.3. Métricas de Calidad Estratégica

Más allá de predecir elecciones humanas, evaluamos si los LLMs generan **mazos objetivamente mejores** que los originales según criterios estratégicos de DeckShop (§2.5).

4.3.1. Mejora Global de Mazos

La Tabla 3 resume el porcentaje de mazos mejorados, cambio promedio en rating total, y cambio en sinergia (di-

Table 3. Mejora de mazos por modelo LLM (configuración `context`). Un mazo se considera mejorado si ΔRating ≥ 0.

Modelo	% Mejorado	ΔRating	ΔSynergy
DeepSeek-V3.1	**58.8%**	-0.11	+0.450
GPT-5-chat	58.7%	-0.03	**+0.613**
LLaMA-3.3-70B	61.9%	-0.38	-0.095
GPT-4o	47.9%	-0.65	+0.113
GPT-3.5-turbo	50.0%	-0.19	-0.688

mensión más relevante para coherencia estratégica).

Insights sobre optimización estratégica: DeepSeek lidera en utilidad práctica: 58.8% de mazos mejorados con ΔSynergy de +0.450, el mejor entre todos los modelos. Confirma especialización en optimización estratégica, priorizando coherencia de roles y sinergias sobre replicación exacta de preferencias.

GPT-5 balancea ambos objetivos: 58.7% de mejora (casi idéntico a DeepSeek) pero con la **mayor sinergia** (+0.613), sugiriendo recomendaciones más coherentes estratégicamente. Combinado con su superior F1-Score (0.221), se posiciona como el modelo más versátil.

GPT-3.5 degrada sistemáticamente sinergia: ΔSynergy de -0.688, el peor de todos. A pesar de "mejorar" 50% de mazos en rating total (probablemente por azar), genera mazos con pobre cohesión: redundancias funcionales, desbalance de roles, combinaciones incoherentes.

LLaMA-3.3 paradoja: Alto % de mejora (61.9%) pero ΔSynergy negativo (-0.095). Recomendaciones erráticas: ocasionalmente acierta con cartas populares que mejoran el rating, pero sin razonamiento estratégico coherente.

4.3.2. Breakdown por Dimensión Estratégica

Para entender qué aspectos mejoran o empeoran cada modelo, analizamos el cambio promedio en las 5 dimensiones de DeckShop (§2.5). La Figura 3 descompone el impacto de cada modelo.

Análisis dimensional detallado: Synergy (amarillo): GPT-5 (+0.613, mejor), DeepSeek (+0.450), GPT-4o (+0.113), LLaMA-3.3 (-0.095), GPT-3.5 (-0.688, peor). Los modelos avanzados generan mazos con sinergias claras y roles complementarios.

Implicación práctica: Trade-off fundamental entre especialización ofensiva y balance estratégico. LLMs avanzados generan mazos más balanceados y sinérgicos, pero potencialmente menos agresivos que preferencias humanas. Valioso en sistemas educativos pero podría no alinearse con jugadores que buscan mazos ultra-ofensivos.

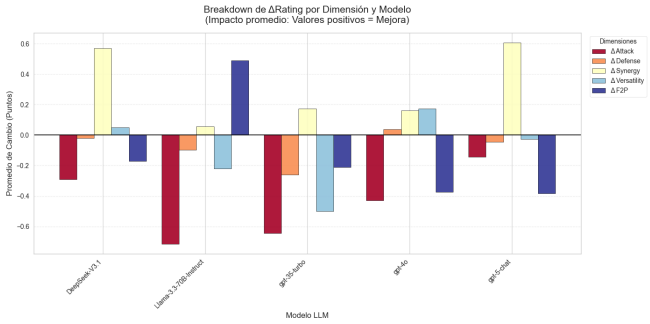

Figure 3. **Detalle de ΔRating por dimensión estratégica y modelo.**

Table 4. Métricas de diversidad y novelty...

Modelo	Coverage (%)	Novelty	Inter-List Div.
Large Language Models			
GPT-5-chat	**38.3%**	5.21	**0.925**
DeepSeek-V3.1	33.9%	5.04	0.906
GPT-4o	33.0%	5.05	0.904
LLaMA-3.3-70B	29.8%	5.17	0.861
GPT-3.5-turbo	26.9%	5.09	0.757
Baselines Tradicionales			
LightFM	12.4%	4.85	0.680
DeepFM	14.2%	4.92	0.695

4.4. Análisis de Diversidad y Novelty

Evaluamos la capacidad de explorar el catálogo más allá de opciones populares (métricas en §2.5). Crucial para evitar filtros burbuja y promover diversidad en el meta-juego.

Hallazgos sobre exploración del catálogo: Correlación positiva capacidad-coverage: Modelos avanzados logran mayor cobertura sin sacrificar coherencia. GPT-5: 38.3% + 0.925 diversity (máximo en ambas). DeepSeek: 33.9% + 0.906. GPT-3.5: 26.9% + 0.757 (penúltimo). LLMs avanzados *razonan* sobre cartas menos obvias sin degradar calidad; modelos débiles recurren a cartas populares por defecto.

Alta inter-list diversity deseable: Valores $\sim$0.9 indican personalización por usuario, no siempre mismas cartas populares. GPT-5 (0.925) y DeepSeek (0.906) destacan.

Novelty moderadamente alta: Valores 5.0-5.2 indican cartas con popularidad *moderada*, evitando ultra-populares (novelty bajo) y casi nunca usadas (novelty muy alto). Estratégicamente sensato.

Baselines conservadores: Recomendaciones genéricas centradas en cartas populares. Baja inter-list diversity (0.68-0.70) indica combinaciones similares a muchos usuarios, limitando exploración.

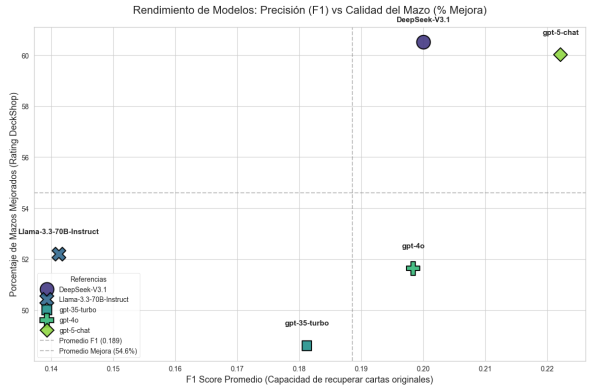

Figure 4. **Trade-off entre precisión de predicción y utilidad estratégica.**

4.5. Trade-offs entre Modelos

Nuestros resultados revelan un **trade-off fundamental** entre: (1) **Precisión de predicción humana** (F1-Score) y (2) **Utilidad estratégica** (% mazos mejorados).

La Figura 4 visualiza este trade-off en el espacio bidimensional precisión vs. utilidad.

Perfiles de modelos identificados: **GPT-5**: Máxima precisión (F1=0.22) + Alta utilidad (60.0%) + Máximo coverage (75.2%). *Caso de uso*: Satisfacción del usuario con exploración activa del catálogo.

DeepSeek-V3.1 : Máxima utilidad (60.5%) + Mejor ΔSynergy (+0.450) + Precisión moderada (F1=0.20). *Caso de uso*: Sistemas educativos/coaching que buscan mejorar decisiones, sugiriendo cartas efectivas aunque menos obvias.

GPT-4o (El Balanceado): Rendimiento intermedio en ambas dimensiones (F1=0.198, 51.6% mejora). *Caso de uso*: Alternativa eficiente en costo cuando GPT-5 no es accesible.

LLaMA-3.3: Alto coverage (79.3%) pero por motivos incorrectos (aleatoriedad). Baja precisión (F1=0.14) y baja validez (25-40%). *Conclusión*: Inadecuado para producción.

GPT-3.5): Degrada sistemáticamente calidad (ΔSynergy=-0.688). Precisión mínima (F1=0.047). *Conclusión*: No usar para tareas estratégicas complejas.

5. Discusión

5.1. Limitaciones del Estudio

1. Cobertura temporal limitada de la API. La API oficial de Clash Royale solo proporciona acceso a las últimas 30 batallas de cada jugador (aproximadamente 7 días de historial). Esto limita nuestra capacidad de:

- Analizar evolución temporal de preferencias (e.g., adaptación al meta tras parches de balance)
- Construir perfiles de usuario de largo plazo (e.g., identificar jugadores que experimentan vs. jugadores conservadores)
- Validar estabilidad de recomendaciones a lo largo del tiempo

2. Evaluación de calidad basada en métricas proxy. Utilizamos DeckShop como *proxy* de calidad estratégica objetiva, pero este servicio tiene limitaciones:

- Los ratings se basan en opinión experta comunitaria, no en resultados empíricos de millones de batallas
- Las 5 dimensiones (Attack, Defense, Synergy, Versatility, F2P) tienen pesos subjetivos
- No captura dinámicas específicas de matchup (e.g., un mazo puede ser excelente contra ciertos arquetipos pero débil contra otros)

Idealmente, evaluaríamos calidad mediante simulaciones de batallas o estudios con usuarios reales jugando los mazos recomendados. Sin embargo, esto requiere infraestructura fuera del alcance de este estudio. DeckShop representa el mejor proxy disponible públicamente, utilizado extensamente por la comunidad competitiva.

3. Restricciones en evaluación masiva de LLMs. Enfrentamos tres restricciones prácticas que limitaron la escala de la evaluación:

- **Rate limits de APIs**: Límites de tokens/minuto impidieron evaluación masiva paralela.
- **CAPTCHA en DeckShop**: Implementado durante el proyecto, requirió pausas de segundos entre solicitudes.

6. Conclusion

6.1. Implicaciones Prácticas

Para **desarrolladores de sistemas recomendadores**:

- Consideren LLMs para dominios donde las sinergias entre ítems son críticas y reglas estratégicas son explicitables
- Usen GPT-5 para maximizar satisfacción del usuario (alta precisión + diversidad)
- Usen DeepSeek para optimizar decisiones estratégicas (máxima utilidad + coherencia)
- Inviertan en prompt engineering: el contexto estratégico puede mejorar rendimiento

6.2. Trabajo Futuro

Este estudio tiene limitaciones que presentan oportunidades para trabajo futuro:

- **Generalización**: Validar en otros juegos estratégicos (MOBAs, auto-battlers, juegos de cartas)
- **Evaluación**: Complementar métricas offline con estudios A/B con usuarios reales
- **Deployment**: Abordar concept drift, cold start, y feedback loops en sistemas en producción
- **Fairness**: Analizar impacto en diversidad del metajuego y equidad entre jugadores

A pesar de estas limitaciones, nuestros resultados establecen que los LLMs representan un avance significativo sobre métodos colaborativos tradicionales para recomendación estratégica.

References

Boonstra, E. and Chen, S. Prompt engineering guide v7: Best practices for large language models. https://ai.google.dev/gemini-api/docs/prompting-strategies, 2025. Accessed: 2024-12-09.

Brown, T. B., Mann, B., Ryder, N., Subbiah, M., Kaplan, J., Dhariwal, P., Neelakantan, A., Shyam, P., Sastry, G., Askell, A., Agarwal, S., Herbert-Voss, A., Krueger, G., Henighan, T., Child, R., Ramesh, A., Ziegler, D. M., Wu, J., Winter, C., Hesse, C., Chen, M., Sigler, E., Litwin, M., Gray, S., Chess, B., Clark, J., Berner, C., McCandlish, S., Radford, A., Sutskever, I., and Amodei, D. Language models are few-shot learners. *Advances in Neural Information Processing Systems*, 33:1877–1901, 2020. URL https://arxiv.org/abs/2005.14165.

DeckShop. Deckshop for clash royale: Deck builder and meta statistics. https://www.deckshop.pro, 2025. Community platform for deck ratings and metagame analysis. Accessed: 2024-12-09.

Guo, H., Tang, R., Ye, Y., Li, Z., and He, X. Deepfm: A factorization-machine based neural network for ctr prediction. In *Proceedings of the 26th International Joint Conference on Artificial Intelligence*, pp. 1725–1731. AAAI Press, 2017.

Koren, Y., Bell, R., and Volinsky, C. Matrix factorization techniques for recommender systems. *Computer*, 42(8): 30–37, 2009.

Kula, M. Metadata embeddings for user and item cold-start recommendations. In *Proceedings of the 2nd Workshop on New Trends on Content-Based Recommender Systems*, pp. 14–21. ACM, 2015.

OpenAI. Gpt-4 technical report. Technical report, OpenAI, 2024. URL https://arxiv.org/abs/2303.08774. Technical report on GPT-4 architecture and capabilities.

Rendle, S. Factorization machines. In *2010 IEEE International Conference on Data Mining*, pp. 995–1000. IEEE, 2010.

Supercell. Clash royale api documentation. https://developer.clashroyale.com, 2025. Official API for accessing player data and game statistics. Accessed: 2024-12-09.

Wei, J., Wang, X., Schuurmans, D., Bosma, M., Ichter, B., Xia, F., Chi, E., Le, Q. V., and Zhou, D. Chain-of-thought prompting elicits reasoning in large language models. *Advances in Neural Information Processing Systems*, 35: 24824–24837, 2022. URL https://arxiv.org/abs/2201.11903.

A. Supplementary Materials

A.1. Prompt Templates

System Prompt:

```
You are an expert Clash Royale strategist. You know deck archetypes,
synergies, win conditions, defensive/offensive roles, and elixir cost
balance. Your task is to COMPLETE a deck to 8 cards, selecting optimal
available cards for the user.
```

Lite Prompt:

```
<Prompt>
  <Inputs>
    <Variable name="CARTAS_DISPONIBLES" type="array[string]">
      $CARTAS_DISPONIBLES%
    </Variable>
    <Variable name="CARTAS_SELECCIONADAS" type="array[string]">
      $CARTAS_SELECCIONADAS%
    </Variable>
  </Inputs>

  <Context>
    $TOP_DECKS%
  </Context>

  <Task>
    Dado CARTAS_DISPONIBLES y CARTAS_SELECCIONADAS, selecciona las mejores
    (8 - |CARTAS_SELECCIONADAS|) cartas en ORDEN DE PRIORIDAD para completar el mazo a 8.
    Si |CARTAS_SELECCIONADAS| == 8, devuelve una seleccin vaca.
  </Task>

  <Constraints>
    <Rule>No propongas cartas fuera de CARTAS_DISPONIBLES.</Rule>
    <Rule>No repitas cartas que ya estan en CARTAS_SELECCIONADAS.</Rule>
    <Rule>El mazo final debe tener exactamente 8 cartas en total, ni mas ni menos.</Rule>
    <Rule>Devuelve en el JSON solamente las cartas nuevas que seleccionaste.</Rule>
  </Constraints>

  <ReasoningGuidelines>
    <Step>Identifica el arquetipo implcito en CARTAS_SELECCIONADAS (p. ej., ciclo,
        beatdown, control, bridge spam, bait).</Step>
    <Step>Detecta vacos de rol y debilidades (p. ej., falta de anti-areo, ausencia de
        hechizo grande/pequeo, carencia de reset).</Step>
    <Step>Prioriza cartas que:
      <Criterion>Potencien la condicin de victoria existente o aadan una complementaria si
          falta.</Criterion>
      <Criterion>Cubran amenazas comunes (enjambres, tanques, win-cons rpidas,
          areo).</Criterion>
      <Criterion>Mejoren matchups sinrgicos sin inflar demasiado el coste
          medio.</Criterion>
    </Step>
  </ReasoningGuidelines>

  <OutputFormat>
    Devuelve nicamente un objeto JSON vlido con este esquema:
    <![CDATA[
    {
      "seleccion": [ "<carta_1>", "<carta_2>", "... hasta N" ]
    }
    ]]>
  </OutputFormat>

  <Style>
    <Rule>Lenguaje conciso y tcnico.</Rule>
```

```
    <Rule>No incluyas texto extra fuera del JSON especificado.</Rule>
  </Style>
</Prompt>
```

Semi-Lite Prompt:

```
<Prompt>
  <Inputs>
    <Variable name="CARTAS_DISPONIBLES" type="array[string]">
      $CARTAS_DISPONIBLES%
    </Variable>
    <Variable name="CARTAS_SELECCIONADAS" type="array[string]">
      $CARTAS_SELECCIONADAS%
    </Variable>
  </Inputs>

  <Context>
    $TOP_DECKS%
  </Context>

  <Task>
    Dado CARTAS_DISPONIBLES y CARTAS_SELECCIONADAS, selecciona las mejores
    (8 - |CARTAS_SELECCIONADAS|) cartas en ORDEN DE PRIORIDAD para completar el mazo a 8.
    Si |CARTAS_SELECCIONADAS| == 8, devuelve una seleccin vaca.
  </Task>

  <Constraints>
    <Rule>No propongas cartas fuera de CARTAS_DISPONIBLES.</Rule>
    <Rule>No repitas cartas que ya estan en CARTAS_SELECCIONADAS.</Rule>
    <Rule>El mazo final debe tener exactamente 8 cartas en total, ni mas ni menos.</Rule>
    <Rule>Devuelve en el JSON solamente las cartas nuevas que seleccionaste.</Rule>
    <Rule>Evita redundancias funcionales claras (p. ej., mltiples hechizos o edificios con
        rol idntico sin justificacin).</Rule>
    <Rule>Favorece sinergias con las cartas ya seleccionadas y cubre roles clave:
      condicin de victoria, respuesta area/tierra, dao en rea, reset/control,
          ciclo/soporte y tanqueo segn el arquetipo que emerja.
    </Rule>
    <Rule>Optimiza el coste medio de elixir mantenindolo razonable para el arquetipo
        detectado.</Rule>
  </Constraints>

  <ReasoningGuidelines>
    <Step>Identifica el arquetipo implcito en CARTAS_SELECCIONADAS (p. ej., ciclo,
        beatdown, control, bridge spam, bait).</Step>
    <Step>Detecta vacos de rol y debilidades (p. ej., falta de anti-areo, ausencia de
        hechizo grande/pequeo, carencia de reset).</Step>
    <Step>Prioriza cartas que:
      <Criterion>Potencien la condicin de victoria existente o aadan una complementaria si
          falta.</Criterion>
      <Criterion>Cubran amenazas comunes (enjambres, tanques, win-cons rpidas,
          areo).</Criterion>
      <Criterion>Mejoren matchups sinrgicos sin inflar demasiado el coste
          medio.</Criterion>
    </Step>
    <Step>Ordena la propuesta por aporte marginal esperado al mazo (1 = ms valiosa).</Step>
  </ReasoningGuidelines>

  <OutputFormat>
    Devuelve nicamente un objeto JSON vlido con este esquema:
    <![CDATA[
    {
      "seleccion": [ "<carta_1>", "<carta_2>", "... hasta N" ]
    }
    ]]>
  </OutputFormat>
```

```
  <EdgeCases>
    <Case name="MazoCompleto">
      Si |CARTAS_SELECCIONADAS| == 8, devuelve:
      <![CDATA[
      { "seleccion": [], "detalle": [], "resumen": "El mazo ya tiene 8 cartas.", "n": 0 }
      ]]>
    </Case>
    <Case name="DisponibilidadInsuficiente">
      Si CARTAS_DISPONIBLES tiene menos de N cartas elegibles, devuelve tantas como sea
          posible y explica en "resumen" la limitacin.
    </Case>
    <Case name="ConflictoDeRoles">
      Si dos opciones compiten por el mismo slot funcional, prioriza la de mayor sinergia
          global y mejor curva; menciona la alternativa en "detalle".
    </Case>
  </EdgeCases>

  <Examples>
    <Example title="Faltan 3 cartas">
      <Input>
        <![CDATA[
        CARTAS_SELECCIONADAS = ["Montapuercos","Mosketeer","Tronco","Can","Espritu de
            Hielo"]
        CARTAS_DISPONIBLES = ["Bola de Fuego","Terremoto","Esqueletos","Arqueras","Brbaros
            de lite"]
        ]]>
      </Input>
      <ExpectedOutputNote>Devuelve 3 cartas ordenadas (8 - |CARTAS_SELECCIONADAS| = 3) con
          justificacin breve y efecto en elixir.</ExpectedOutputNote>
    </Example>
  </Examples>

  <Style>
    <Rule>Lenguaje conciso y tcnico.</Rule>
    <Rule>No incluyas texto extra fuera del JSON especificado.</Rule>
  </Style>
</Prompt>
```

Full Prompt:

```
<Prompt>
  <Inputs>
    <Variable name="CARTAS_DISPONIBLES" type="array[string]">
      $CARTAS_DISPONIBLES%
    </Variable>
    <Variable name="CARTAS_SELECCIONADAS" type="array[string]">
      $CARTAS_SELECCIONADAS%
    </Variable>
  </Inputs>

  <Context>
    $TOP_DECKS%
  </Context>

  <Task>
    Dado CARTAS_DISPONIBLES y CARTAS_SELECCIONADAS, selecciona las mejores
    (8 - |CARTAS_SELECCIONADAS|) cartas en ORDEN DE PRIORIDAD para completar el mazo a 8.
    Si |CARTAS_SELECCIONADAS| == 8, devuelve una seleccin vaca.
  </Task>

  <Constraints>
    <Rule>No propongas cartas fuera de CARTAS_DISPONIBLES.</Rule>
    <Rule>No repitas cartas que ya estan en CARTAS_SELECCIONADAS.</Rule>
    <Rule>El mazo final debe tener exactamente 8 cartas en total, ni mas ni menos.</Rule>
```

```
  <Rule>Devuelve en el JSON solamente las cartas nuevas que seleccionaste.</Rule>
  <Rule>Evita redundancias funcionales claras (p. ej., mltiples hechizos o edificios con
      rol idntico sin justificacin).</Rule>
  <Rule>Favorece sinergias con las cartas ya seleccionadas y cubre roles clave:
    condicin de victoria, respuesta area/tierra, dao en rea, reset/control,
        ciclo/soporte y tanqueo segn el arquetipo que emerja.
  </Rule>
  <Rule>Optimiza el coste medio de elixir mantenindolo razonable para el arquetipo
      detectado.</Rule>
</Constraints>

<ReasoningGuidelines>
  <Step>Identifica el arquetipo implcito en CARTAS_SELECCIONADAS (p. ej., ciclo,
      beatdown, control, bridge spam, bait).</Step>
  <Step>Detecta vacos de rol y debilidades (p. ej., falta de anti-areo, ausencia de
      hechizo grande/pequeo, carencia de reset).</Step>
  <Step>Prioriza cartas que:
    <Criterion>Potencien la condicin de victoria existente o aadan una complementaria si
        falta.</Criterion>
    <Criterion>Cubran amenazas comunes (enjambres, tanques, win-cons rpidas,
        areo).</Criterion>
    <Criterion>Mejoren matchups sinrgicos sin inflar demasiado el coste
        medio.</Criterion>
  </Step>
  <Step>Ordena la propuesta por aporte marginal esperado al mazo (1 = ms valiosa).</Step>
</ReasoningGuidelines>

<OutputFormat>
  Devuelve nicamente un objeto JSON vlido con este esquema:
  <![CDATA[
  {
    "seleccion": [ "<carta_1>", "<carta_2>", "... hasta N" ],
    "detalle": [
      {
        "carta": "<nombre>",
        "costo_elixir": "<costo_de_elixir>",
        "rol": "<rol_principal>",
        "sinergiasCon": ["<carta_existente_1>", "..."],
        "cubreDebilidad": ["<debilidad_1>", "..."],
        "impactoElixir": {"antes": <float>, "despues": <float>}
      }
    ],
    "resumen": "1-2 frases con el arquetipo final y el plan de juego",
    "n": <N> // N = 8 - |CARTAS_SELECCIONADAS|
  }
  ]]>
</OutputFormat>

<EdgeCases>
  <Case name="MazoCompleto">
    Si |CARTAS_SELECCIONADAS| == 8, devuelve:
    <![CDATA[
    { "seleccion": [], "detalle": [], "resumen": "El mazo ya tiene 8 cartas.", "n": 0 }
    ]]>
  </Case>
  <Case name="DisponibilidadInsuficiente">
    Si CARTAS_DISPONIBLES tiene menos de N cartas elegibles, devuelve tantas como sea
        posible y explica en "resumen" la limitacin.
  </Case>
  <Case name="ConflictoDeRoles">
    Si dos opciones compiten por el mismo slot funcional, prioriza la de mayor sinergia
        global y mejor curva; menciona la alternativa en "detalle".
  </Case>
</EdgeCases>
```

```
  <Examples>
    <Example title="Faltan 3 cartas">
      <Input>
        <![CDATA[
        CARTAS_SELECCIONADAS = ["Montapuercos","Mosketeer","Tronco","Can","Espritu de
            Hielo"]
        CARTAS_DISPONIBLES = ["Bola de Fuego","Terremoto","Esqueletos","Arqueras","Brbaros
            de lite"]
        ]]>
      </Input>
      <ExpectedOutputNote>Devuelve 3 cartas ordenadas (8 - |CARTAS_SELECCIONADAS| = 3) con
          justificacin breve y efecto en elixir.</ExpectedOutputNote>
    </Example>
  </Examples>

  <Style>
    <Rule>Lenguaje conciso y tcnico.</Rule>
    <Rule>No incluyas texto extra fuera del JSON especificado.</Rule>
  </Style>
</Prompt>
```

Context Example:

```
<Context>
  <TopDecks>
    <Note>Top 5 Decks para usar con cada una de las CARTAS_DISPONIBLES</Note>

    <CardGroup name="Wizard">
      <Deck>
        <Card>Freeze</Card>
        <Card>Balloon</Card>
        <Card>Skeletons</Card>
        <Card>Valkyrie</Card>
        <Card>Zap</Card>
        <Card>Mega Minion</Card>
        <Card>Wizard</Card>
        <Card>Ice Spirit</Card>
      </Deck>
      <Deck>
        <Card>Tornado</Card>
        <Card>Bowler</Card>
        <Card>Knight</Card>
        <Card>Inferno Dragon</Card>
        <Card>Graveyard</Card>
        <Card>Freeze</Card>
        <Card>Wizard</Card>
        <Card>Mother Witch</Card>
      </Deck>
      <Deck>
        <Card>The Log</Card>
        <Card>Elixir Collector</Card>
        <Card>Skeletons</Card>
        <Card>Golem</Card>
        <Card>Prince</Card>
        <Card>Mega Minion</Card>
        <Card>Poison</Card>
        <Card>Wizard</Card>
      </Deck>
      <Deck>
        <Card>Baby Dragon</Card>
        <Card>Valkyrie</Card>
        <Card>Mega Knight</Card>
        <Card>Tornado</Card>
        <Card>The Log</Card>
        <Card>Wizard</Card>
```

```
    <Card>Firecracker</Card>
    <Card>Pekka</Card>
  </Deck>
  <Deck>
    <Card>Golem</Card>
    <Card>Barbarian Barrel</Card>
    <Card>Tornado</Card>
    <Card>Skeleton King</Card>
    <Card>Wizard</Card>
    <Card>Night Witch</Card>
    <Card>Goblin Cage</Card>
    <Card>Zap</Card>
  </Deck>
</CardGroup>

<CardGroup name="Skeleton Army">
  <Deck>
    <Card>Skeleton Army</Card>
    <Card>Dart Goblin</Card>
    <Card>Mortar</Card>
    <Card>Minion Horde</Card>
    <Card>Skeleton King</Card>
    <Card>Arrows</Card>
    <Card>Miner</Card>
    <Card>Cannon Cart</Card>
  </Deck>
  <Deck>
    <Card>Skeleton Army</Card>
    <Card>Zap</Card>
    <Card>Freeze</Card>
    <Card>Minions</Card>
    <Card>Dart Goblin</Card>
    <Card>Ice Golem</Card>
    <Card>Cannon</Card>
    <Card>Balloon</Card>
  </Deck>
  <Deck>
    <Card>Goblin Drill</Card>
    <Card>Wall Breakers</Card>
    <Card>Tesla</Card>
    <Card>Bomber</Card>
    <Card>Rocket</Card>
    <Card>Fire Spirit</Card>
    <Card>Golden Knight</Card>
    <Card>Skeleton Army</Card>
  </Deck>
  <Deck>
    <Card>Skeleton Army</Card>
    <Card>Minions</Card>
    <Card>Inferno Tower</Card>
    <Card>Fire Spirit</Card>
    <Card>Ice Golem</Card>
    <Card>Mega Minion</Card>
    <Card>The Log</Card>
    <Card>Mortar</Card>
  </Deck>
  <Deck>
    <Card>Balloon</Card>
    <Card>Giant</Card>
    <Card>Skeleton Army</Card>
    <Card>Minions</Card>
    <Card>Arrows</Card>
    <Card>Skeletons</Card>
    <Card>Minion Horde</Card>
    <Card>Freeze</Card>
```

```
  </Deck>
</CardGroup>

<CardGroup name="Inferno Tower">
  <Deck>
    <Card>Electro Wizard</Card>
    <Card>Miner</Card>
    <Card>Goblin Gang</Card>
    <Card>Battle Ram</Card>
    <Card>Knight</Card>
    <Card>Poison</Card>
    <Card>Inferno Tower</Card>
    <Card>The Log</Card>
  </Deck>
  <Deck>
    <Card>Mini Pekka</Card>
    <Card>Guards</Card>
    <Card>Inferno Tower</Card>
    <Card>Electro Dragon</Card>
    <Card>Arrows</Card>
    <Card>Inferno Dragon</Card>
    <Card>Zap</Card>
    <Card>Bats</Card>
  </Deck>
  <Deck>
    <Card>Electro Wizard</Card>
    <Card>X Bow</Card>
    <Card>The Log</Card>
    <Card>Skeletons</Card>
    <Card>Knight</Card>
    <Card>Fireball</Card>
    <Card>Ice Spirit</Card>
    <Card>Inferno Tower</Card>
  </Deck>
  <Deck>
    <Card>Knight</Card>
    <Card>Princess</Card>
    <Card>Ice Spirit</Card>
    <Card>Rocket</Card>
    <Card>Goblin Gang</Card>
    <Card>Inferno Tower</Card>
    <Card>The Log</Card>
    <Card>Goblin Drill</Card>
  </Deck>
  <Deck>
    <Card>Ice Spirit</Card>
    <Card>Inferno Tower</Card>
    <Card>Ice Wizard</Card>
    <Card>Ice Golem</Card>
    <Card>Poison</Card>
    <Card>Miner</Card>
    <Card>Bats</Card>
    <Card>Zap</Card>
  </Deck>
</CardGroup>

<CardGroup name="Mini P.E.K.K.A">
  <Deck>
    <Card>Executioner</Card>
    <Card>Witch</Card>
    <Card>Royal Ghost</Card>
    <Card>Mini Pekka</Card>
    <Card>Arrows</Card>
    <Card>Knight</Card>
    <Card>Valkyrie</Card>
```

```
      <Card>Poison</Card>
    </Deck>
    <Deck>
      <Card>Spear Goblins</Card>
      <Card>Fireball</Card>
      <Card>Archers</Card>
      <Card>Goblins</Card>
      <Card>Arrows</Card>
      <Card>Mini Pekka</Card>
      <Card>Baby Dragon</Card>
      <Card>Giant</Card>
    </Deck>
    <Deck>
      <Card>The Log</Card>
      <Card>Ice Golem</Card>
      <Card>Ice Spirit</Card>
      <Card>Mini Pekka</Card>
      <Card>Princess</Card>
      <Card>Hog Rider</Card>
      <Card>Rocket</Card>
      <Card>Bats</Card>
    </Deck>
    <Deck>
      <Card>Tornado</Card>
      <Card>Ice Golem</Card>
      <Card>Executioner</Card>
      <Card>Mini Pekka</Card>
      <Card>Bats</Card>
      <Card>Hog Rider</Card>
      <Card>Lightning</Card>
      <Card>The Log</Card>
    </Deck>
    <Deck>
      <Card>Sparky</Card>
      <Card>Rage</Card>
      <Card>Mini Pekka</Card>
      <Card>Goblin Giant</Card>
      <Card>Zap</Card>
      <Card>Phoenix</Card>
      <Card>Golden Knight</Card>
      <Card>Spear Goblins</Card>
    </Deck>
  </CardGroup>

 </TopDecks>
</Context>
```

Multi-Step Prompt:

```
<Prompt>
 <Inputs>
  <Variable name="CARTAS_DISPONIBLES" type="array[string]">
   $CARTAS_DISPONIBLES%
  </Variable>
  <Variable name="CARTAS_SELECCIONADAS" type="array[string]">
   $CARTAS_SELECCIONADAS%
  </Variable>
 </Inputs>

 <Context>
  $TOP_DECKS%
 </Context>

 <Task>
  Dado CARTAS_DISPONIBLES y CARTAS_SELECCIONADAS, selecciona las mejores
```

```
  (8 - |CARTAS_SELECCIONADAS|) cartas en ORDEN DE PRIORIDAD para completar el mazo a 8.
  Si |CARTAS_SELECCIONADAS| == 8, devuelve una seleccin vaca.

  Debes trabajar esta tarea en MLTIPLES PASOS:
  1. SELECCIN INICIAL: Realiza tu primera seleccin de cartas
  2. REVISIN Y VERIFICACIN: Revisa crticamente tu seleccin
  3. REFINAMIENTO: Corrige cualquier problema encontrado durante la revisin
</Task>

<Constraints>
  <Rule>No propongas cartas fuera de CARTAS_DISPONIBLES.</Rule>
  <Rule>No repitas cartas que ya estan en CARTAS_SELECCIONADAS.</Rule>
  <Rule>El mazo final debe tener exactamente 8 cartas en total, ni mas ni menos.</Rule>
  <Rule>Devuelve en el JSON solamente las cartas nuevas que seleccionaste.</Rule>
  <Rule>Evita redundancias funcionales claras (p. ej., mltiples hechizos o edificios con
      rol idntico sin justificacin).</Rule>
  <Rule>Favorece sinergias con las cartas ya seleccionadas y cubre roles clave:
    condicin de victoria, respuesta area/tierra, dao en rea, reset/control,
        ciclo/soporte y tanqueo segn el arquetipo que emerja.
  </Rule>
  <Rule>Optimiza el coste medio de elixir mantenindolo razonable para el arquetipo
      detectado.</Rule>
</Constraints>

<MultiStepProcess>
  <Step number="1" name="SeleccionInicial">
    <Description>Realiza tu seleccin inicial de cartas basndote en las guas de
        razonamiento.</Description>
    <Actions>
      <Action>Identifica el arquetipo implcito en CARTAS_SELECCIONADAS.</Action>
      <Action>Detecta vacos de rol y debilidades.</Action>
      <Action>Selecciona cartas que aborden estos vacos manteniendo sinergia.</Action>
      <Action>Calcula el impacto esperado en el coste de elixir.</Action>
    </Actions>
    <Output>Documenta tu razonamiento inicial y seleccin en el campo
        "cadenaDeRazonamiento".</Output>
  </Step>

  <Step number="2" name="RevisionYVerificacion">
    <Description>Revisa crticamente tu seleccin inicial para verificar que tenga
        sentido.</Description>
    <Checklist>
      <Check>Estn todas las cartas seleccionadas dentro de CARTAS_DISPONIBLES?</Check>
      <Check>Hay cartas duplicadas (con CARTAS_SELECCIONADAS o dentro de tu
          seleccin)?</Check>
      <Check>El mazo final tiene exactamente 8 cartas en total?</Check>
      <Check>Hay redundancias funcionales que necesiten justificacin?</Check>
      <Check>Las cartas cubren adecuadamente roles clave (condicin de victoria,
          anti-areo, hechizos, etc.)?</Check>
      <Check>El coste de elixir es razonable para el arquetipo?</Check>
      <Check>Hay mejores alternativas disponibles en CARTAS_DISPONIBLES?</Check>
      <Check>Las cartas funcionan bien juntas sinrgicamente?</Check>
    </Checklist>
    <Output>Documenta los hallazgos de tu revisin en el campo "revision", incluyendo
        cualquier problema identificado.</Output>
  </Step>

  <Step number="3" name="Refinamiento">
    <Description>Si se encontraron problemas en el Paso 2, corrgelos. Si no hay
        problemas, confirma la seleccin.</Description>
    <Actions>
      <Action>Si se encontraron problemas: Reemplaza cartas problemticas con mejores
          alternativas de CARTAS_DISPONIBLES.</Action>
      <Action>Si no hay problemas: Confirma que la seleccin inicial es ptima.</Action>
      <Action>Re-verifica que la seleccin refinada cumple todas las
```

```
        restricciones.</Action>
    </Actions>
    <Output>Documenta cualquier cambio realizado y el razonamiento final en el campo
        "refinamiento".</Output>
  </Step>
</MultiStepProcess>

<ReasoningGuidelines>
  <Step>Identifica el arquetipo implcito en CARTAS_SELECCIONADAS (p. ej., ciclo,
      beatdown, control, bridge spam, bait).</Step>
  <Step>Detecta vacos de rol y debilidades (p. ej., falta de anti-areo, ausencia de
      hechizo grande/pequeo, carencia de reset).</Step>
  <Step>Prioriza cartas que:
    <Criterion>Potencien la condicin de victoria existente o aadan una complementaria si
        falta.</Criterion>
    <Criterion>Cubran amenazas comunes (enjambres, tanques, win-cons rpidas,
        areo).</Criterion>
    <Criterion>Mejoren matchups sinrgicos sin inflar demasiado el coste
        medio.</Criterion>
  </Step>
  <Step>Ordena la propuesta por aporte marginal esperado al mazo (1 = ms valiosa).</Step>
</ReasoningGuidelines>

<OutputFormat>
  Devuelve nicamente un objeto JSON vlido con este esquema:
  <![CDATA[
  {
    "seleccion": [ "<carta_1>", "<carta_2>", "... hasta N" ],
    "detalle": [
      {
        "carta": "<nombre>",
        "costo_elixir": "<costo_de_elixir>",
        "rol": "<rol_principal>",
        "sinergiasCon": ["<carta_existente_1>", "..."],
        "cubreDebilidad": ["<debilidad_1>", "..."],
        "impactoElixir": {"antes": <float>, "despues": <float>}
      }
    ],
    "resumen": "1-2 frases con el arquetipo final y el plan de juego",
    "n": <N>, // N = 8 - |CARTAS_SELECCIONADAS|
    "cadenaDeRazonamiento": {
      "seleccionInicial": {
        "arquetipoIdentificado": "<arquetipo>",
        "vaciosDeRol": ["<vacio_1>", "<vacio_2>", "..."],
        "razonamiento": "Explicacin detallada de por qu cada carta fue seleccionada
            inicialmente",
        "cartasIniciales": [ "<carta_1>", "<carta_2>", "..." ],
        "analisisElixir": "Anlisis del impacto en el coste de elixir"
      },
      "revision": {
        "resultadosVerificacion": {
          "todasCartasDisponibles": <boolean>,
          "sinDuplicados": <boolean>,
          "conteoCorrecto": <boolean>,
          "sinRedundancias": <boolean>,
          "rolesCubiertos": <boolean>,
          "elixirRazonable": <boolean>,
          "verificacionSinergia": <boolean>
        },
        "problemasEncontrados": [
          {
            "problema": "<descripcion>",
            "severidad": "<critico|mayor|menor>",
            "cartasAfectadas": ["<carta_1>", "..."]
          }
```

```
      ],
      "notasRevision": "Evaluacin general de la seleccin inicial"
    },
    "refinamiento": {
      "cambiosRealizados": <boolean>,
      "cartasReemplazadas": [
        {
          "removida": "<nombre_carta>",
          "razon": "<por_que_removida>",
          "reemplazadaPor": "<nombre_carta>",
          "razon": "<por_que_mejor>"
        }
      ],
      "razonamientoFinal": "Explicacin de la seleccin final y por qu es ptima",
      "confianza": "<alta|media|baja>"
    }
  }
 }
 ]]>
</OutputFormat>

<EdgeCases>
 <Case name="MazoCompleto">
  Si |CARTAS_SELECCIONADAS| == 8, devuelve:
  <![CDATA[
  {
    "seleccion": [],
    "detalle": [],
    "resumen": "El mazo ya tiene 8 cartas.",
    "n": 0,
    "cadenaDeRazonamiento": {
      "seleccionInicial": {"razonamiento": "El mazo ya est completo."},
      "revision": {"resultadosVerificacion": {}, "problemasEncontrados": [],
          "notasRevision": "No se requiere accin."},
      "refinamiento": {"cambiosRealizados": false, "razonamientoFinal": "Mazo
          completo.", "confianza": "alta"}
    }
  }
  ]]>
 </Case>
 <Case name="DisponibilidadInsuficiente">
  Si CARTAS_DISPONIBLES tiene menos de N cartas elegibles, devuelve tantas como sea
      posible y explica la limitacin en "resumen" y
      "cadenaDeRazonamiento.refinamiento.razonamientoFinal".
 </Case>
 <Case name="ConflictoDeRoles">
  Si dos opciones compiten por el mismo slot funcional, prioriza la de mayor sinergia
      global y mejor curva; menciona la alternativa en "detalle" y documenta la decisin
      en "cadenaDeRazonamiento".
 </Case>
</EdgeCases>

<Examples>
 <Example title="Faltan 3 cartas">
  <Input>
   <![CDATA[
   CARTAS_SELECCIONADAS = ["Montapuercos","Mosketeer","Tronco","Can","Espritu de
       Hielo"]
   CARTAS_DISPONIBLES = ["Bola de Fuego","Terremoto","Esqueletos","Arqueras","Brbaros
       de lite"]
   ]]>
  </Input>
  <ExpectedOutputNote>
   Devuelve 3 cartas con cadena de razonamiento completa mostrando:
   - Razonamiento de seleccin inicial (identificando arquetipo de ciclo, necesidad de
```

```
          hechizo, etc.)
      - Hallazgos de revisin (verificando que todas las restricciones se cumplan)
      - Cualquier refinamiento realizado (si la seleccin inicial tena problemas)
    </ExpectedOutputNote>
  </Example>
</Examples>

<Style>
  <Rule>Lenguaje conciso y tcnico.</Rule>
  <Rule>S exhaustivo en la cadena de razonamiento - muestra tu proceso completo de
      razonamiento.</Rule>
  <Rule>No incluyas texto extra fuera del JSON especificado.</Rule>
</Style>
</Prompt>
```

English Full Prompt:

```
<Prompt>
  <Inputs>
    <Variable name="AVAILABLE_CARDS" type="array[string]">
      $CARTAS_DISPONIBLES%
    </Variable>
    <Variable name="SELECTED_CARDS" type="array[string]">
      $CARTAS_SELECCIONADAS%
    </Variable>
  </Inputs>

  <Context>
    $TOP_DECKS%
  </Context>

  <Task>
    Given AVAILABLE_CARDS and SELECTED_CARDS, select the best
    (8 - |SELECTED_CARDS|) cards in PRIORITY ORDER to complete the deck to 8.
    If |SELECTED_CARDS| == 8, return an empty selection.
  </Task>

  <Constraints>
    <Rule>Do not propose cards outside of AVAILABLE_CARDS.</Rule>
    <Rule>Do not repeat cards that are already in SELECTED_CARDS.</Rule>
    <Rule>The final deck must have exactly 8 cards in total, no more, no less.</Rule>
    <Rule>Return in the JSON only the new cards you selected.</Rule>
    <Rule>Avoid clear functional redundancies (e.g., multiple spells or buildings with
        identical roles without justification).</Rule>
    <Rule>Favor synergies with already selected cards and cover key roles:
      win condition, air/ground response, area damage, reset/control, cycle/support and
          tanking according to the emerging archetype.
    </Rule>
    <Rule>Optimize the average elixir cost keeping it reasonable for the detected
        archetype.</Rule>
  </Constraints>

  <ReasoningGuidelines>
    <Step>Identify the implicit archetype in SELECTED_CARDS (e.g., cycle, beatdown,
        control, bridge spam, bait).</Step>
    <Step>Detect role gaps and weaknesses (e.g., lack of anti-air, absence of large/small
        spell, lack of reset).</Step>
    <Step>Prioritize cards that:
      <Criterion>Enhance the existing win condition or add a complementary one if
          missing.</Criterion>
      <Criterion>Cover common threats (swarms, tanks, fast win-cons, air).</Criterion>
      <Criterion>Improve synergistic matchups without inflating the average cost too
          much.</Criterion>
    </Step>
    <Step>Order the proposal by expected marginal contribution to the deck (1 = most
```

```
      valuable).</Step>
  </ReasoningGuidelines>

  <OutputFormat>
    Return only a valid JSON object with this schema:
    <![CDATA[
    {
      "selection": [ "<card_1>", "<card_2>", "... up to N" ],
      "detail": [
        {
          "card": "<name>",
          "elixir_cost": "<elixir_cost>",
          "role": "<main_role>",
          "synergiesWith": ["<existing_card_1>", "..."],
          "coversWeakness": ["<weakness_1>", "..."],
          "elixirImpact": {"before": <float>, "after": <float>}
        }
      ],
      "summary": "1-2 sentences with the final archetype and game plan",
      "n": <N> // N = 8 - |SELECTED_CARDS|
    }
    ]]>
  </OutputFormat>

  <EdgeCases>
    <Case name="CompleteDeck">
      If |SELECTED_CARDS| == 8, return:
      <![CDATA[
      { "selection": [], "detail": [], "summary": "The deck already has 8 cards.", "n": 0 }
      ]]>
    </Case>
    <Case name="InsufficientAvailability">
      If AVAILABLE_CARDS has fewer than N eligible cards, return as many as possible and
          explain the limitation in "summary".
    </Case>
    <Case name="RoleConflict">
      If two options compete for the same functional slot, prioritize the one with greater
          global synergy and better curve; mention the alternative in "detail".
    </Case>
  </EdgeCases>

  <Examples>
    <Example title="Missing 3 cards">
      <Input>
        <![CDATA[
        SELECTED_CARDS = ["Hog Rider","Musketeer","The Log","Cannon","Ice Spirit"]
        AVAILABLE_CARDS = ["Fireball","Earthquake","Skeletons","Archers","Elite Barbarians"]
        ]]>
      </Input>
      <ExpectedOutputNote>Returns 3 cards ordered (8 - |SELECTED_CARDS| = 3) with brief
          justification and elixir effect.</ExpectedOutputNote>
    </Example>
  </Examples>

  <Style>
    <Rule>Concise and technical language.</Rule>
    <Rule>Do not include extra text outside the specified JSON.</Rule>
  </Style>
</Prompt>
```

Multi-Step English Prompt:

```
<Prompt>
  <Inputs>
    <Variable name="AVAILABLE_CARDS" type="array[string]">
```

```
    $CARTAS_DISPONIBLES%
  </Variable>
  <Variable name="SELECTED_CARDS" type="array[string]">
    $CARTAS_SELECCIONADAS%
  </Variable>
</Inputs>

<Context>
  $TOP_DECKS%
</Context>

<Task>
  Given AVAILABLE_CARDS and SELECTED_CARDS, select the best
  (8 - |SELECTED_CARDS|) cards in PRIORITY ORDER to complete the deck to 8.
  If |SELECTED_CARDS| == 8, return an empty selection.

  You must work through this task in MULTIPLE STEPS:
  1. INITIAL SELECTION: Make your first card selection
  2. REVIEW & VERIFICATION: Critically review your selection
  3. REFINEMENT: Fix any issues found during review
</Task>

<Constraints>
  <Rule>Do not propose cards outside of AVAILABLE_CARDS.</Rule>
  <Rule>Do not repeat cards that are already in SELECTED_CARDS.</Rule>
  <Rule>The final deck must have exactly 8 cards in total, no more, no less.</Rule>
  <Rule>Return in the JSON only the new cards you selected.</Rule>
  <Rule>Avoid clear functional redundancies (e.g., multiple spells or buildings with
      identical roles without justification).</Rule>
  <Rule>Favor synergies with already selected cards and cover key roles:
    win condition, air/ground response, area damage, reset/control, cycle/support and
        tanking according to the emerging archetype.
  </Rule>
  <Rule>Optimize the average elixir cost keeping it reasonable for the detected
      archetype.</Rule>
</Constraints>

<MultiStepProcess>
  <Step number="1" name="InitialSelection">
    <Description>Make your initial card selection based on the reasoning
        guidelines.</Description>
    <Actions>
      <Action>Identify the implicit archetype in SELECTED_CARDS.</Action>
      <Action>Detect role gaps and weaknesses.</Action>
      <Action>Select cards that address these gaps while maintaining synergy.</Action>
      <Action>Calculate the expected elixir cost impact.</Action>
    </Actions>
    <Output>Document your initial reasoning and selection in the "chainOfThought"
        field.</Output>
  </Step>

  <Step number="2" name="ReviewAndVerification">
    <Description>Critically review your initial selection to verify it makes
        sense.</Description>
    <Checklist>
      <Check>Are all selected cards within AVAILABLE_CARDS?</Check>
      <Check>Are there any duplicate cards (with SELECTED_CARDS or within your
          selection)?</Check>
      <Check>Does the final deck have exactly 8 cards total?</Check>
      <Check>Are there any functional redundancies that need justification?</Check>
      <Check>Do the cards properly cover key roles (win condition, anti-air, spells,
          etc.)?</Check>
      <Check>Is the elixir cost reasonable for the archetype?</Check>
      <Check>Are there better alternatives available in AVAILABLE_CARDS?</Check>
      <Check>Do the cards work well together synergistically?</Check>
```

```
    </Checklist>
    <Output>Document your review findings in the "review" field, including any issues
        identified.</Output>
  </Step>

  <Step number="3" name="Refinement">
    <Description>If issues were found in Step 2, fix them. If no issues, confirm the
        selection.</Description>
    <Actions>
      <Action>If issues found: Replace problematic cards with better alternatives from
          AVAILABLE_CARDS.</Action>
      <Action>If no issues: Confirm that the initial selection is optimal.</Action>
      <Action>Re-verify the refined selection meets all constraints.</Action>
    </Actions>
    <Output>Document any changes made and the final reasoning in the "refinement"
        field.</Output>
  </Step>
</MultiStepProcess>

<ReasoningGuidelines>
  <Step>Identify the implicit archetype in SELECTED_CARDS (e.g., cycle, beatdown,
      control, bridge spam, bait).</Step>
  <Step>Detect role gaps and weaknesses (e.g., lack of anti-air, absence of large/small
      spell, lack of reset).</Step>
  <Step>Prioritize cards that:
    <Criterion>Enhance the existing win condition or add a complementary one if
        missing.</Criterion>
    <Criterion>Cover common threats (swarms, tanks, fast win-cons, air).</Criterion>
    <Criterion>Improve synergistic matchups without inflating the average cost too
        much.</Criterion>
  </Step>
  <Step>Order the proposal by expected marginal contribution to the deck (1 = most
      valuable).</Step>
</ReasoningGuidelines>

<OutputFormat>
  Return only a valid JSON object with this schema:
  <![CDATA[
  {
    "selection": [ "<card_1>", "<card_2>", "... up to N" ],
    "detail": [
      {
        "card": "<name>",
        "elixir_cost": "<elixir_cost>",
        "role": "<main_role>",
        "synergiesWith": ["<existing_card_1>", "..."],
        "coversWeakness": ["<weakness_1>", "..."],
        "elixirImpact": {"before": <float>, "after": <float>}
      }
    ],
    "summary": "1-2 sentences with the final archetype and game plan",
    "n": <N>, // N = 8 - |SELECTED_CARDS|
    "chainOfThought": {
      "initialSelection": {
        "archetypeIdentified": "<archetype>",
        "roleGaps": ["<gap_1>", "<gap_2>", "..."],
        "reasoning": "Detailed explanation of why each card was initially selected",
        "initialCards": [ "<card_1>", "<card_2>", "..." ],
        "elixirAnalysis": "Analysis of elixir cost impact"
      },
      "review": {
        "verificationResults": {
          "allCardsAvailable": <boolean>,
          "noDuplicates": <boolean>,
          "correctCount": <boolean>,
```

```
        "noRedundancies": <boolean>,
        "rolesCovered": <boolean>,
        "elixirReasonable": <boolean>,
        "synergyCheck": <boolean>
      },
      "issuesFound": [
        {
          "issue": "<description>",
          "severity": "<critical|major|minor>",
          "affectedCards": ["<card_1>", "..."]
        }
      ],
      "reviewNotes": "Overall assessment of the initial selection"
    },
    "refinement": {
      "changesMade": <boolean>,
      "cardsReplaced": [
        {
          "removed": "<card_name>",
          "reason": "<why_removed>",
          "replacedWith": "<card_name>",
          "reason": "<why_better>"
        }
      ],
      "finalReasoning": "Explanation of the final selection and why it's optimal",
      "confidence": "<high|medium|low>"
    }
  }
}
]]>
</OutputFormat>

<EdgeCases>
  <Case name="CompleteDeck">
    If |SELECTED_CARDS| == 8, return:
    <![CDATA[
    {
      "selection": [],
      "detail": [],
      "summary": "The deck already has 8 cards.",
      "n": 0,
      "chainOfThought": {
        "initialSelection": {"reasoning": "Deck is already complete."},
        "review": {"verificationResults": {}, "issuesFound": [], "reviewNotes": "No
            action needed."},
        "refinement": {"changesMade": false, "finalReasoning": "Deck complete.",
            "confidence": "high"}
      }
    }
    ]]>
  </Case>
  <Case name="InsufficientAvailability">
    If AVAILABLE_CARDS has fewer than N eligible cards, return as many as possible and
        explain the limitation in "summary" and
        "chainOfThought.refinement.finalReasoning".
  </Case>
  <Case name="RoleConflict">
    If two options compete for the same functional slot, prioritize the one with greater
        global synergy and better curve; mention the alternative in "detail" and document
        the decision in "chainOfThought".
  </Case>
</EdgeCases>

<Examples>
  <Example title="Missing 3 cards">
```

```
    <Input>
      <![CDATA[
      SELECTED_CARDS = ["Hog Rider","Musketeer","The Log","Cannon","Ice Spirit"]
      AVAILABLE_CARDS = ["Fireball","Earthquake","Skeletons","Archers","Elite Barbarians"]
      ]]>
    </Input>
    <ExpectedOutputNote>
      Returns 3 cards with complete chain of thought showing:
      - Initial selection reasoning (identifying cycle archetype, need for spell, etc.)
      - Review findings (verifying all constraints met)
      - Any refinements made (if initial selection had issues)
    </ExpectedOutputNote>
  </Example>
</Examples>

<Style>
  <Rule>Concise and technical language.</Rule>
  <Rule>Be thorough in the chain of thought - show your complete reasoning
      process.</Rule>
  <Rule>Do not include extra text outside the specified JSON.</Rule>
</Style>
</Prompt>
```

XXXX Prompt:

A.2. Additional Results